

DATA SCIENCE CAPSTONE PROJECT REPORT

SpaceX Falcon 9 First Stage Landing Prediction

Prepared by: Kunal Singh

Date: 13 September 2026

IBM Data Science Professional Certificate — Final Capstone

Project focus: Data collection, data wrangling, exploratory analysis, SQL, geospatial visualization, dashboard design, and classification modelling.

1. Executive Summary

This capstone investigates whether a SpaceX Falcon 9 first stage can land successfully after launch. Reusing the first stage can reduce launch cost, making landing success an important operational and business outcome. The project applies an end-to-end data science workflow using launch records, exploratory analysis, SQL, Folium, Plotly Dash, and supervised machine learning.

The workflow collects launch information, transforms nested or inconsistent fields into a tabular dataset, checks missing values and duplicates, engineers a binary landing-success target, and explores relationships involving launch site, payload mass, orbit, booster configuration, and mission period.

The predictive task is binary classification. Successful landing is encoded as 1 and unsuccessful landing as 0. Candidate algorithms include Logistic Regression, Support Vector Machine, Decision Tree, and K-Nearest Neighbours. The final model must be evaluated using the actual test-set accuracy, precision, recall, F1-score, and confusion matrix from the completed notebook.

Conclusion: landing success should be analysed as a combined effect of mission context, payload characteristics, launch site, and vehicle configuration. A transparent model can support exploratory risk analysis, but it cannot replace engineering, weather, safety, or mission-control decisions.

Submission evidence note: The example charts and example metric values in this report show the required structure. Replace them with actual notebook exports and actual metrics before final submission.

2. Introduction

SpaceX uses reusable launch technology in which the Falcon 9 first stage may return to Earth after launch. Landing success is a useful outcome variable because it reflects whether the booster recovery objective was achieved. Understanding the factors associated with success provides a practical application of data science to aerospace operations.

Project objective

To collect, clean, explore, visualize, and model SpaceX Falcon 9 launch data in order to understand factors associated with first-stage landing success and develop a classification model that predicts the landing outcome.

Research questions

- How does landing success vary across launch sites and mission periods?
- What relationships exist between payload mass, orbit, booster version, and landing outcome?
- Which features are most useful for predicting landing success?
- How can SQL, Folium, and Plotly Dash communicate the findings clearly?

Scope and limitations

The analysis uses historical public or course-provided records. It is intended for educational pattern discovery. It does not include every engineering, weather, vehicle-health, or real-time operational variable.

3. Data Collection and Data Wrangling Methodology

Data source

The project uses the SpaceX launch dataset used in the IBM Applied Data Science Capstone workflow. Data may be collected from the SpaceX REST API or loaded from the course-provided CSV generated during the notebook workflow. The final repository README should state the exact source URL or file name used.

Collection procedure

- Request or load launch records from the approved data source.
- Extract nested JSON fields into a pandas DataFrame.
- Combine launch, booster, payload, launch-site, and landing information where available.
- Save the collected dataset in the repository data/ directory.

Wrangling steps

- Inspect columns, data types, duplicates, and missing values.
- Convert dates into consistent datetime values and derive year or month features.
- Standardize categorical values such as orbit, launch site, booster version, and landing outcome.
- Create a binary Class target from the landing outcome.
- Document imputation or removal rules for missing values.
- Encode categorical variables and scale numeric variables when required.
- Split the dataset into training and testing sets using a fixed random state.

Representative Python workflow

```
import pandas as pd

df = pd.read_csv("data/spacex_launch_data.csv")
df = df.drop_duplicates()
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
df["Year"] = df["Date"].dt.year

# Match this field to the actual completed dataset.
```

```
df["Class"] = df["Landing_Outcome"].apply(
    lambda x: 1 if str(x).lower() in ["successful", "success"] else 0
)

df.to_csv("data/spacex_cleaned.csv", index=False)
```

4. EDA and Data Visualization Methodology

Exploratory Data Analysis is used to understand the dataset before modelling. The analysis combines descriptive statistics, frequency tables, grouped summaries, and visualizations. Every visual should be accompanied by an interpretation that connects it to a research question.

- Use count plots to compare successful and unsuccessful landings.
- Use grouped bar charts to compare success rates by launch site, orbit, and booster version.
- Use scatter plots to examine payload mass against landing outcome or predicted probability.
- Use time-based plots to observe changes across launch years.
- Use correlation or feature-importance analysis cautiously because association does not prove causation.

5. EDA with Visualization

The following visuals demonstrate the evidence format required in the final presentation. They are illustrative examples and must be replaced with actual exported figures from the completed notebook.

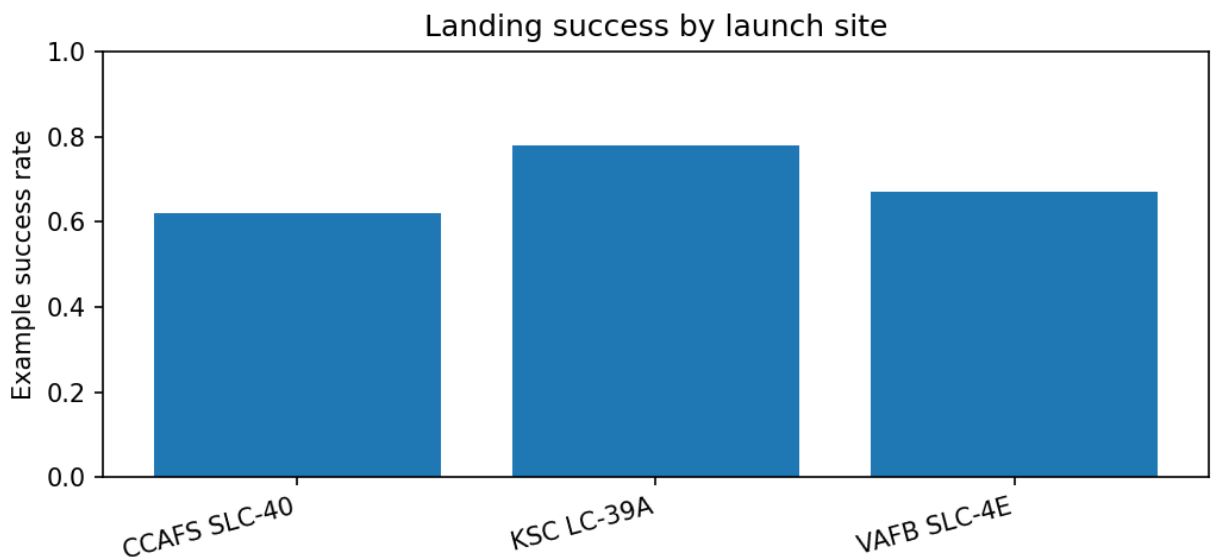


Figure 1 interpretation: A launch-site comparison can reveal whether success rates differ across locations. Differences may reflect trajectory, landing infrastructure, mission mix, or sample size.

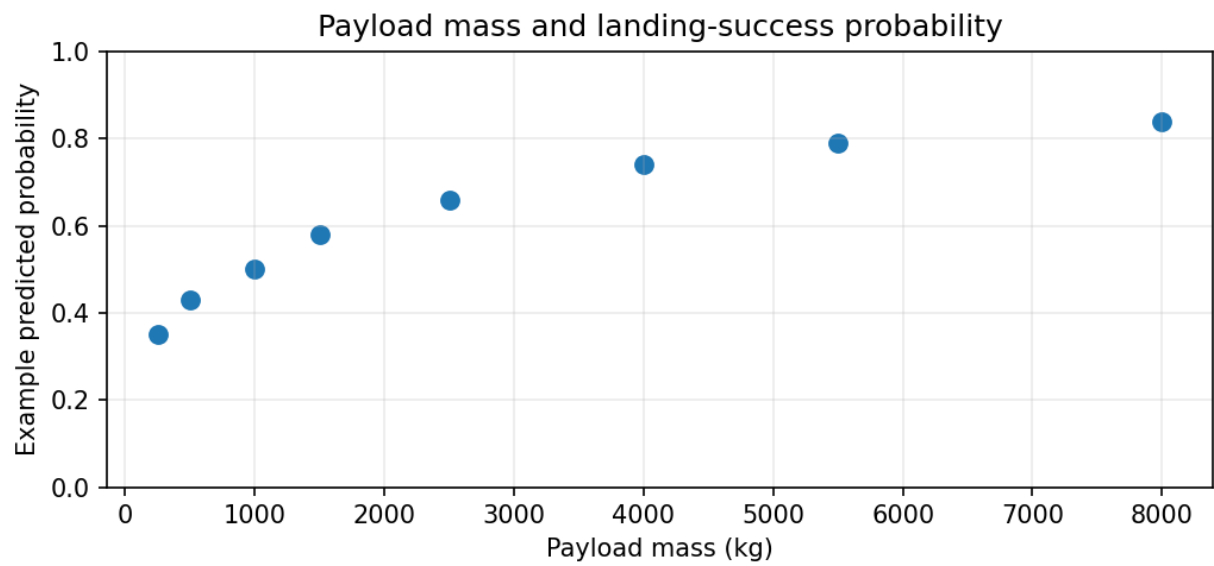


Figure 2 interpretation: Payload mass can be compared with landing outcome or predicted probability. Payload should be interpreted together with orbit and booster configuration.

6. EDA with SQL: Queries and Results

SQL analysis demonstrates the ability to answer reproducible analytical questions using SELECT, WHERE, GROUP BY, ORDER BY, aggregate functions, and joins. The final presentation should include screenshots of executed queries and their actual output tables.

Example table: SPACEX_LAUNCHES

```
-- Count launches by launch site
SELECT Launch_Site, COUNT(*) AS Total_Launches
FROM SPACEX_LAUNCHES
GROUP BY Launch_Site
ORDER BY Total_Launches DESC;

-- Count successful and unsuccessful landings
SELECT Class, COUNT(*) AS Number_of_Launches
FROM SPACEX_LAUNCHES
GROUP BY Class;

-- Average payload by orbit
SELECT Orbit, AVG(Payload_Mass_kg) AS Average_Payload
FROM SPACEX_LAUNCHES
GROUP BY Orbit
ORDER BY Average_Payload DESC;

-- Success rate by launch site
SELECT Launch_Site,
       AVG(Class) * 100 AS Success_Rate_Percent
FROM SPACEX_LAUNCHES
GROUP BY Launch_Site
ORDER BY Success_Rate_Percent DESC;
```

Required presentation evidence

- Show each SQL query in a readable code block.
- Show the actual result table below each query.
- Explain at least one insight from each result.
- Use the same column names and row counts as the completed SQL notebook.

Interpretation: Grouping by launch site allows comparison of launch volume and landing success. Percentages should always be considered together with the number of observations.

7. Folium Map

Folium is used to create an interactive map of SpaceX launch locations. Each marker can display the launch-site name, coordinates, total launches, and success rate.

```
import folium

launch_map = folium.Map(
    location=[28.5, -80.6],
    zoom_start=4
)

sites = [
    {"name": "CCAFS SLC-40", "lat": 28.5619, "lon": -80.5774},
    {"name": "KSC LC-39A", "lat": 28.6083, "lon": -80.6041},
    {"name": "VAFB SLC-4E", "lat": 34.6328, "lon": -120.6108},
]

for site in sites:
    folium.Marker(
        [site["lat"], site["lon"]],
        popup=site["name"],
        tooltip=site["name"]
    ).add_to(launch_map)

launch_map.save("folium_launch_sites.html")
```

Map interpretation: The map adds geographic context to the statistical comparison and helps the audience connect launch-site results to physical locations. Include a screenshot of the rendered map in the final presentation.

8. Plotly Dash Dashboard

The dashboard communicates the analysis interactively. A suitable dashboard can include launch-site and orbit filters, a success-rate chart, a payload relationship chart, and a filtered launch-data table.

Suggested dashboard layout

- Header: SpaceX Falcon 9 Landing Prediction.
- Controls: launch site, orbit, booster version, and payload range.
- Main chart: landing success rate by selected category.
- Secondary chart: payload mass versus landing outcome.
- Data table: filtered launch records.
- Summary cards: total launches, successful landings, and success percentage.

Representative Dash structure

```
from dash import Dash, dcc, html
import plotly.express as px

app = Dash(__name__)

summary = df.groupby("Launch_Site", as_index=False)["Class"].mean()
fig = px.bar(summary, x="Launch_Site", y="Class",
             title="Landing success rate by launch site")

app.layout = html.Div([
    html.H1("SpaceX Falcon 9 Landing Prediction"),
    dcc.Graph(figure=fig)
])

if __name__ == "__main__":
    app.run(debug=True)
```

Required evidence: Include a screenshot of the running dashboard or the actual dashboard output. A heading alone is not sufficient.

9. Predictive Analysis: Classification

The predictive task is binary classification. The target variable indicates whether the first stage landed successfully. Depending on the completed dataset, predictors may include launch site, orbit, payload mass, booster version, grid-fin status, legs status, landing-pad category, and time-related features.

Modelling workflow

- Select relevant predictors and define the target class.
- Preprocess numeric and categorical variables.
- Encode categorical variables and scale numeric features where appropriate.
- Split the data into training and testing sets.
- Train multiple classification models.
- Evaluate accuracy, precision, recall, F1-score, and the confusion matrix.
- Compare models and select the model with the most appropriate test performance.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

X = df[feature_columns]
y = df["Class"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```

model = Pipeline([
    ("scaler", StandardScaler(with_mean=False)),
    ("classifier", LogisticRegression(max_iter=1000))
])

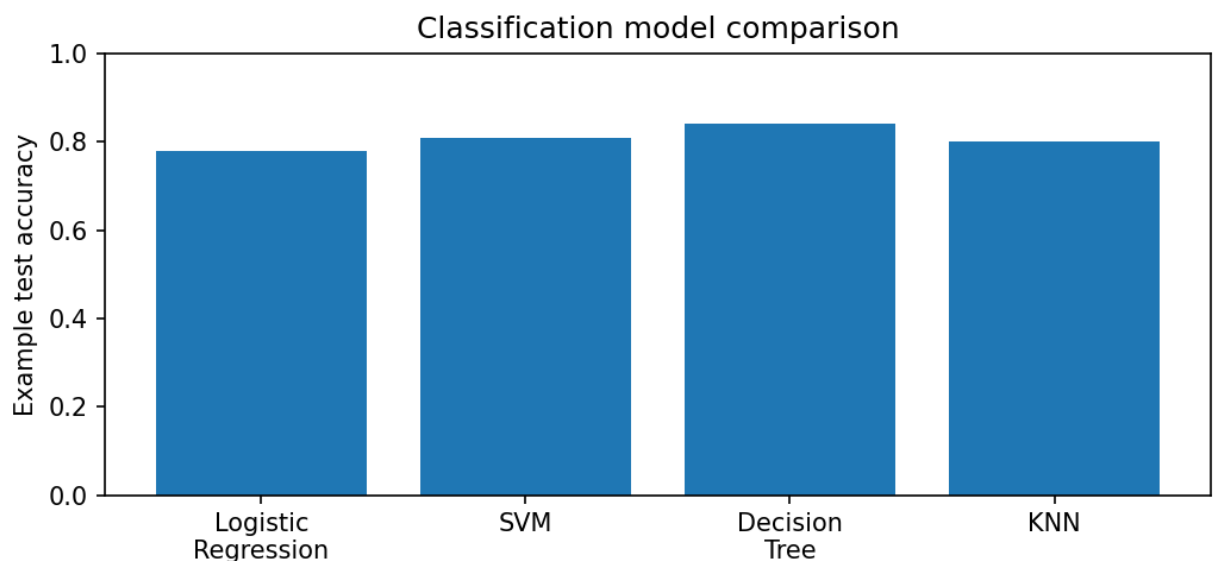
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print(accuracy_score(y_test, predictions))
print(classification_report(y_test, predictions))

```

10. Predictive Analysis Results

This section must contain the actual evaluation results produced by the completed notebook. The example chart below shows the required presentation format; replace it with the actual model comparison generated from the test set.



Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	Actual	Actual	Actual	Actual
Support Vector Machine	Actual	Actual	Actual	Actual
Decision Tree	Actual	Actual	Actual	Actual
K-Nearest Neighbours	Actual	Actual	Actual	Actual

Interpretation instructions: Identify the strongest model using actual test-set metrics. Discuss false positives and false negatives using the confusion matrix. Explain class balance, overfitting risk, and the limitations of the available features.

Insert actual confusion matrix here: The final PDF should contain the confusion-matrix image exported from the notebook.

11. Conclusion

This capstone demonstrates an end-to-end data science workflow using an aerospace dataset. Data collection and wrangling transform raw launch records into an analysis-ready table. EDA identifies patterns across launch sites, payloads, orbits, booster configurations, and time. SQL provides reproducible analysis, Folium adds geographic context, and Plotly Dash communicates findings interactively.

The classification stage converts landing outcome into a binary target and compares several supervised-learning algorithms. Model selection should be based on actual test-set performance rather than training performance alone. The model can reveal patterns associated with landing success, but it cannot capture every engineering, environmental, or operational factor.

Recommendations

- Maintain a reproducible data pipeline with documented source files and cleaning steps.
- Track performance as new launch records become available.
- Use class-aware metrics and a confusion matrix rather than accuracy alone.
- Add reliable weather and mission-condition variables if available.
- Use the model as an analytical aid, not as a standalone safety decision system.

Limitations

- Historical records may contain missing or inconsistently recorded fields.
- The sample size is limited compared with industrial machine-learning datasets.
- Historical mission patterns may not represent future missions equally.
- Observed relationships should not automatically be interpreted as causal.

12. Creativity and Innovative Insights

Creative elements

- Combining statistical charts, SQL evidence, a geographic map, and an interactive dashboard.
- Using a clear narrative: business problem → data → evidence → model → recommendation.
- Adding interpretations beneath visuals instead of presenting charts without explanation.
- Comparing multiple evaluation metrics rather than reporting only one score.

Potential insights to verify using actual outputs

- Landing performance may differ across launch sites because mission mix and infrastructure differ.
- Payload mass should be interpreted jointly with orbit and booster configuration.
- Performance may change over time as operational experience and recovery technology develop.
- An interactive dashboard can help users examine how outcomes vary across launch conditions.

These are analytical hypotheses until verified against the actual dataset and notebook outputs.

13. GitHub Repository and Reproducibility

Repository URL: <https://github.com/kunalji-j/data-science-capstone-kunal-singh>

Recommended repository structure

```
data-science-capstone-kunal-singh/  
|  
|-- README.md  
|-- Kunal_Singh_Data_Collection_API.ipynb  
|-- Data_Wrangling.ipynb  
|-- EDA_Visualization.ipynb
```

```

|-- SQL_Analysis.ipynb
|-- Folium_Map.ipynb
|-- Predictive_Analysis.ipynb
|-- Plotly_Dash/
|   |-- app.py
|-- data/
|   |-- spacex_launch_data.csv
|   |-- spacex_cleaned.csv
|-- Kunal_Singh_Final_Presentation.pdf

```

Repository checklist

- The repository is public or accessible to the grader.
- The README includes the objective, tools, dataset source, and run instructions.
- The notebooks contain executed cells, outputs, charts, and explanations.
- The SQL notebook includes queries and visible result tables.
- The Folium notebook includes a displayed or saved interactive map.
- The Dash folder includes app.py or documented dashboard evidence.
- The predictive notebook includes actual metrics and a confusion matrix.
- The PDF contains the real GitHub URL and matches the repository contents.

14. Final Evidence Checklist

Evidence item	Before submission
Executive Summary with findings	■
Introduction and research questions	■
Data source and collection method	■
Wrangling steps and output	■
EDA charts with interpretations	■
SQL queries and actual results	■
Folium map screenshot	■
Plotly Dash screenshot or working app	■
Classification workflow	■
Actual model metrics	■
Actual confusion matrix	■
Conclusion, limitations, recommendations	■
Creativity and innovative insights	■
Working GitHub URL	■

Prepared for academic submission by Kunal Singh — 13 September 2026.